UNIT II: JavaScript

Q1. Explain Event Handling in JavaScript with suitable example.

Event handling in JavaScript allows us to execute code in response to user actions such as clicks, keypresses, form submissions, etc. JavaScript provides multiple ways to attach event handlers to HTML elements.

Common Events:

- `onclick` – when an element is clicked
- `onmouseover` – when the mouse is over an element
- `onchange` – when a form element changes
- `onsubmit` – when a form is submitted

Ways to Handle Events:

1. **Inline in HTML:** `<button onclick="showMsg()">Click Me</button>`

2. **Using JavaScript:**

```
<button id="btn">Click</button>
<script>
  document.getElementById("btn").onclick = function() {
    alert("Button Clicked");
  };
</script>
```

3. **Using addEventListener:**

```
document.getElementById("btn").addEventListener("click", function() {
  alert("Clicked via addEventListener");
});
```

Benefits:

- Separates behavior from structure (cleaner code)
- Allows multiple events to be attached to the same element

Event handling adds interactivity to web pages and is essential for user-driven functionality.

Q2. Identify function object in JavaScript? Also how to create function objects that uses Function Constructor and new Operator.

In JavaScript, **functions are first-class objects**. This means:

- Functions can be stored in variables
- Passed as arguments
- Returned from other functions
- Assigned properties

Every function in JavaScript is actually an instance of the Function object.

Creating Functions in JavaScript

1. Function Declaration (Standard way)

```
function greet() {
  return "Hello";
}
```

2. Function Expression (Assigned to a variable)

```
const greet = function() {
  return "Hello";
};
```

3. Using Function Constructor with new Operator

You can create functions dynamically using the Function constructor.

Syntax: `const func = new Function("a", "b", "return a + b;");`

Usage: `let sum = func(5, 10);  // sum = 15`

Explanation:

- "a" and "b" are parameter names
- "return a + b;" is the function body

Q3. How to use Array in JavaScript? Also explain types of Array.

An **array** is a special variable that can hold multiple values at once. In JavaScript, arrays are dynamic and can store elements of different types.

Creating Arrays:

```
let fruits = ["Apple", "Banana", "Mango"];
let numbers = new Array(1, 2, 3, 4);
```

Accessing Elements: `console.log(fruits[0]); // Apple`

Common Operations:

- `push()` – Add at end
- `pop()` – Remove from end
- `shift()` – Remove from beginning
- `unshift()` – Add at beginning
- `length` – Get number of elements

Types of Arrays in JavaScript:

1. **Single-Dimensional Array**: A basic list of elements: `let cities = ["Pune", "Mumbai", "Nashik"];`
2. **Multi-Dimensional Array**: An array of arrays (used like a matrix): `let matrix = [[1, 2], [3, 4]];`

Arrays are versatile and essential for managing lists, collections, and datasets in JavaScript.

Q4. Explain Math and Date object in JavaScript

The Math object is a built-in object that has properties and methods for mathematical constants and functions.

Method	Description
`Math.round(x)`	Rounds x to the nearest integer
`Math.floor(x)`	Rounds x down
`Math.ceil(x)`	Rounds x up
`Math.max(a,b)`	Returns the largest number
`Math.min(a,b)`	Returns the smallest number
`Math.random()`	Returns a random number (0–1)
`Math.pow(a,b)`	a to the power of b

`Math.sqrt(x)`	Square root of x

The Date object represents date and time.

Creating Date:

```
let now = new Date();            // current date and time
let specific = new Date("2025-08-07");
```

Useful Date Methods:

Method	Description
`getFullYear()`	Gets the 4-digit year
`getMonth()`	Gets the month (0-11)
`getDate()`	Gets the day of the month (1-31)
`getDay()`	Gets the weekday (0–6, Sunday=0)
`getHours()`	Gets the hour (0-23)

Q5. Differentiate between server-side scripting and client-side scripting.

Feature	Client-Side Scripting	Server-Side Scripting
Execution Location	Runs in the user's browser	Runs on the web server
Languages	JavaScript, HTML, CSS	PHP, Python, Node.js, ASP.NET, Java
Purpose	User interface, interactivity	Database access, authentication, business logic
Speed	Fast (no server communication needed)	Slower due to server response time
Security	Less secure (visible in browser)	More secure (code is hidden on server)
Access to Files/DB	Cannot directly access server files or DB	Can read/write to files and databases

- **Client-Side:** Validating a form using JavaScript before submission.
- **Server-Side:** Storing user registration data into a database.

Both are essential. Client-side scripting enhances user experience, while server-side scripting handles backend processing and data management.

Q6. What do you understand by use of JavaScript for Form validation?

Form validation in JavaScript ensures that inputs are correct, complete before the form is submitted to the server.

1. **Client-Side Validation** (using JavaScript): Happens in the browser before submission Faster feedback to users Example: Checking if required fields are filled
2. **Server-Side Validation**: Done after the form is submitted. More secure but slower

Common Validations in JavaScript:

- Required field check
- Email format validation
- Password length check
- Matching passwords
- Numeric or date input

Example:

```
<form onsubmit="return validateForm()">
  <input type="text" id="name">
  <input type="submit">
</form>

<script>
  function validateForm() {
```

```
    let name = document.getElementById("name").value;
    if (name === "") {
      alert("Name is required");
      return false;
    }
    return true;
  }
</script>
```

JavaScript validation improves user experience by preventing incomplete or incorrect submissions, reducing server load and response time.

Q7. Differentiate between Functions and Events with the help of example

Aspect	Function	Event
Definition	A block of reusable code that performs a task	An action or occurrence recognized by JavaScript
Execution	Called manually or programmatically	Triggered automatically when something happens
Purpose	Performs a specific task	Responds to user interaction or browser action
Example	`myFunction()`	onclick, onload, onchange, etc.

Function Example:

```
function greet() {
  alert("Hello!");
}
greet(); // Manually called
```

Event Example:

```
<button onclick="greet()">Click Me</button>
<script>
  function greet() {
    alert("Button Clicked!");
  }
</script>
```

- **Functions** are building blocks used to perform logic.
- **Events** are triggers that call functions when something happens (e.g., click, load, focus).

Q8. Briefly explain the various types of dialog boxes in JavaScript with example.

1. Alert Box: Used to display a message. Only an "OK" button is shown. Used for notifications.

```
alert("Form submitted successfully!");
```

2. Confirm Box: Asks user for confirmation. "OK" and "Cancel" buttons. Returns true if OK, false if Cancel.

```
let result = confirm("Are you sure?");
if (result) {
  // proceed
}
```

3. Prompt Box: Asks user to input a value. Has a textbox, "OK", and "Cancel". Returns input string or null.

```
let name = prompt("Enter your name:");
alert("Hello, " + name);
```

These dialog boxes are useful for user interaction and validation, but are often replaced by modern UI components for better control and design.

Q9. State and explain session cookie.

A **session cookie** is a temporary cookie that is stored in the browser's memory only for the duration of the user's session on a website.

Key Characteristics:

- **Temporary**: Deleted when the browser is closed.
- **No expiration date**: Unlike persistent cookies, session cookies do not have an expiry time.
- **Used for**: Maintaining login state. Storing temporary preferences. Tracking user activity during a single session.

Example: `document.cookie = "username=Ajit";` This sets a session cookie with the name username.

When a user logs into a website, a session cookie can be created to remember that the user is authenticated until they log out or close the browser. Session cookies help provide a smooth user experience by maintaining temporary data, especially for authenticated sessions.

Q10. Describe the following events with their attribute and tags in JavaScript.

These events are essential for handling user interaction and improving the dynamic behavior of web pages.

a. Focus: Triggered when an

- **Attribute**: onfocus

```
<input type="text" onfocus="highlight(this)">
<script>
  function highlight(el) {
    el.style.background = "lightyellow";
  }
</script>
```

b. Click: Triggered when an element is clicked.

- **Attribute**: onclick

```
<button onclick="sayHello()">Click Me</button>
<script>
  function sayHello() {
    alert("Hello!");
  }
</script>
```

c. Load: Triggered when the page or image is fully l

- **Attribute**: onload

```
<body onload="initPage()">
<script>
  function initPage() {
    alert("Page Loaded");
  }
</script>
```

d. Submit: Triggered when a form is submitted.

```
<form onsubmit="return validateForm()">
  <input type="text" required>
  <button type="submit">Submit</button>
</form>
<script>
  function validateForm() {
    alert("Form Submitted");
    return true;
  }
</script>
```